

SPE Amplifier Remote Control

User Guide

For SPE Expert 1.3K-FA / 1.5K-FA / 2K-FA HF Amplifiers

Original script by **OH2GEK**

Modernized and extended by **VU2CPL**

Version	2.0 (Python 3)
Platform	Raspberry Pi / Linux / macOS
Interface	Web browser (any device)
Connection	USB / RS-232 Serial
Protocol	SPE App Programmer's Guide Rev 1.1

Table of Contents

1. Introduction
2. Requirements
3. Installation
4. Configuration
5. Starting the Server
6. Web Interface Guide
7. Power On / Off Control
8. Controls Reference
9. SPE Serial Protocol Reference
10. Running as a System Service
11. WebSocket API
12. Troubleshooting

1. Introduction

SPE Remote Control is a modern Python 3 application that allows you to monitor and control SPE Expert HF amplifiers remotely from any web browser. It communicates with the amplifier via USB or RS-232 serial connection and serves a real-time web dashboard over your local network.

Key Features:

- **Power On/Off** - remote power control via DTR hardware line (on) and serial command 0x0A (off).
- **Full SPE protocol** - all 20 commands from the official Application Programmer's Guide Rev 1.1.
- **Self-contained** - single process serves both the WebSocket API and web UI. No separate web server needed.
- **Multi-client** - multiple browsers and devices can monitor the amplifier simultaneously.
- **Real-time gauges** - SWR, drain current, PA temperature, and voltage displayed with animated arc gauges.
- **Responsive design** - works on desktop, tablet, and mobile screens.
- **Auto-reconnect** - both WebSocket and serial connections reconnect automatically on failure.
- **Async I/O** - non-blocking serial communication using pyserial-asyncio for reliable performance.
- **Configurable** - YAML configuration file for serial port, baud rate, polling intervals, and more.

2. Requirements

Hardware:

- Raspberry Pi (any model - Pi 2, 3, 4, 5, or Zero 2 W) or any Linux/macOS computer
- SPE Expert amplifier (1.3K-FA, 1.5K-FA, 2K-FA, or compatible model)
- USB cable (with FTDI adapter) or RS-232 serial connection to the amplifier
- Network connection (Ethernet or Wi-Fi)

Software:

- Python 3.9 or newer (included in Raspberry Pi OS)
- python3-venv package (for virtual environment)
- Git (for cloning the repository)

Python Dependencies (installed automatically):

Package	Version	Purpose
tornado	>= 6.0	Web server and WebSocket framework
pyserial-asyncio	>= 0.6	Async serial port communication
pyyaml	>= 6.0	YAML configuration file parsing

3. Installation

Step 1: Clone the Repository

```
git clone https://github.com/vu2cpl/spe-remote.git
cd spe-remote
```

Step 2: Run Setup

```
./setup.sh
```

The setup script creates a Python virtual environment and installs all dependencies. On Raspberry Pi OS, it will also install python3-venv if needed.

Step 3: Add User to dialout Group

To access serial ports without sudo, add your user to the dialout group:

```
sudo usermod -aG dialout $USER
```

Log out and log back in for the group change to take effect.

4. Configuration

All settings are in the **config.yaml** file in the project root directory. Edit this file to match your setup.

```
serial:
  port: /dev/serial/by-id/usb-FTDI_FT232R_USB_UART_...
  baudrate: 115200
  timeout: 1.0

server:
  port: 8888
  host: "0.0.0.0"

polling:
  tx_interval: 0.2
  idle_interval: 1.0
  heartbeat: 15

logging:
  level: INFO
```

Configuration Options:

Setting	Default	Description
serial.port	/dev/ttyUSB0	Serial port path to amplifier
serial.baudrate	115200	Serial baud rate
server.port	8888	HTTP/WebSocket listen port
server.host	0.0.0.0	Listen address (0.0.0.0 = all interfaces)
polling.tx_interval	0.2	Poll interval during TX (seconds)
polling.idle_interval	1.0	Poll interval during RX/Standby
polling.heartbeat	15	Force state broadcast interval
logging.level	INFO	Log verbosity (DEBUG/INFO/WARNING/ERROR)

Finding Your Serial Port:

```
# List USB serial devices
ls /dev/serial/by-id/

# Or check dmesg after plugging in the USB cable
dmesg | grep ttyUSB
```

Tip: Use /dev/serial/by-id/... paths instead of /dev/ttyUSB0. The by-id paths are stable across reboots and don't change if other USB devices are connected.

5. Starting the Server

Start with the run script:

```
./run.sh
```

Or start directly with the virtual environment Python:

```
venv/bin/python server.py
```

To use a custom config file:

```
venv/bin/python server.py /path/to/custom-config.yaml
```

On successful startup you will see:

```
[INFO] spe: Server listening on http://0.0.0.0:8888/  
[INFO] spe: Serial port: /dev/ttyUSB0 @ 115200 baud  
[INFO] spe.serial_handler: Connecting...  
[INFO] spe.serial_handler: Serial connected
```

Open your browser and navigate to **`http://<pi-ip-address>:8888/`**

6. Web Interface Guide

The web interface provides a real-time view of the amplifier status with interactive controls.

Connection Indicator

The green/red dot at the top shows WebSocket connection status. If disconnected, the client will automatically attempt to reconnect with increasing delays.

Power Output Display

The large power bar shows the current output power in watts (0-1500W). The gradient color changes from green through cyan and yellow to red as power increases. SWR is displayed in the top-right corner.

Gauges

Four semi-circular arc gauges display real-time readings:

Gauge	Range	Warning Zone
SWR	1:1 to 1:3.5	Above 1:2.0 (orange), above 1:2.8 (red)
Drain Current	0 - 60 A	Above 42 A (orange), above 51 A (red)
PA Temperature	0 - 80 C	Above 60 C (orange), above 64 C (red)
Voltage	40 - 60 V	Below 44 V or above 55 V (warning)

Status Chips

A row of status chips shows: TX/RX status (red pulse during TX, green during RX), current band, antenna number, input number, and power level setting.

Alert Bar

Warning and error messages from the amplifier are displayed at the bottom in an orange (warning) or red (error) bar. Power on/off results show in green (success) or red (failure) and auto-clear after 4 seconds.

7. Power On / Off Control

The web interface includes dedicated Power ON and Power OFF buttons at the top of the control area. These use different mechanisms as defined by the SPE protocol.

Power ON

Power ON is performed by toggling the DTR (Data Terminal Ready) hardware line on the USB-serial adapter. There is no serial data command for powering on the amplifier. The DTR/RTS toggle sequence is based on the original `power_spe_on.py` script by OH2GEK:

```
DTR=1 -> DTR=0 -> RTS=1 -> wait 1s -> DTR=1 -> RTS=0
```

After the sequence, the amplifier takes 3 to 4.5 seconds to start up. When DTR is held high, the amplifier shows a 'POWER SWITCH HELD BY REMOTE' warning and the front-panel power switch is overridden by the remote.

Power OFF

Power OFF uses the official SPE serial command **SWITCH OFF (0x0A)** as documented in the SPE Application Programmer's Guide Rev 1.1. This is equivalent to pressing the front-panel OFF button.

```
Packet: 0x55 0x55 0x55 0x01 0x0A 0x0A
```

Action	Method	Mechanism
Power ON	DTR hardware toggle	USB-serial DTR/RTS line sequence
Power OFF	Serial command 0x0A	SWITCH OFF per SPE protocol

Safety:

Both Power ON and Power OFF buttons require a browser confirmation dialog before executing. The buttons show a shimmer animation while the command is being processed and display a success or error result in the alert bar.

8. Controls Reference

Button	Command	Description
POWER ON	DTR toggle	Powers on the amplifier via hardware DTR line. Confirmation required.
POWER OFF	0x0A	Sends SWITCH OFF command to power down the amplifier. Confirmation required.
Operate	0x0D	Toggles between Operate and Standby modes. Button highlights cyan in Operate.
ANT	0x04	Cycles through available TX antenna outputs (ANT 1, ANT 2, etc.).
TUNE	0x09	Initiates the internal antenna tuning unit (ATU) cycle.
INPUT	0x01	Switches between available input connectors (IN 1, IN 2).
POWER	0x0B	Cycles through power levels: Low, Mid, High.
BAND -	0x02	Steps down one band (e.g. 40m to 80m).
BAND +	0x03	Steps up one band (e.g. 40m to 30m).

Note: All connected clients can send commands. In a multi-client setup, coordinate with other operators to avoid conflicting commands.

9. SPE Serial Protocol Reference

Based on the **SPE Application Programmer's Guide Rev 1.1** for Expert 1.3K-FA / 1.5K-FA / 2K-FA. Communication is asynchronous, 8N1, up to 115200 baud (auto-adapts).

Packet Format

```
0x55 0x55 0x55 [CNT] [DATA...] [CHK]
```

CNT = number of data bytes (checksum excluded)

CHK = modulo-256 sum of DATA bytes

For single-byte commands: CNT=0x01, CHK=DATA

Complete Command Set

Hex	Command	Description
0x01	INPUT	Toggle input port
0x02	BAND -	Band down
0x03	BAND +	Band up
0x04	ANTENNA	Cycle TX antenna
0x05	L-	ATU inductance minus
0x06	L+	ATU inductance plus
0x07	C-	ATU capacitance minus
0x08	C+	ATU capacitance plus
0x09	TUNE	Start ATU tuning
0x0A	SWITCH OFF	Power OFF the amplifier
0x0B	POWER	Toggle power level (L/M/H)
0x0C	DISPLAY	Display toggle
0x0D	OPERATE	Toggle Operate/Standby
0x0E	CAT	CAT mode
0x0F	LEFT ARROW	Menu navigation left
0x10	RIGHT ARROW	Menu navigation right
0x11	SET	Menu enter/set
0x82	BACKLIGHT ON	Turn display backlight on
0x83	BACKLIGHT OFF	Turn display backlight off
0x90	STATUS	Request status string

Status String Fields

The amplifier returns a 67-character ASCII comma-separated status string with 19 fields:

Field	Length	Contents
ID	3	20K (2K-FA) or 13K (1.3K-FA)
Stby/Oper	1	S (Standby) or O (Operate)
RX/TX	1	R (Receive) or T (Transmit)
Memory	1	A, B, or x
Input	1	1 or 2
Band	2	00 (160m) to 11 (4m)
TX Ant + ATU	2	0-6, suffix: t/b/a
RX Ant	2	Antenna number or 0r
Power Level	1	L (Low), M (Mid), H (High)
Output Power	4	Watts (0000 on RX)
SWR ATU	5	VSWR before ATU
SWR ANT	5	VSWR at antenna
V PA	4	Supply voltage
I PA	4	Drain current
Temp Upper	3	Heatsink temp (C or F)
Temp Lower	3	Lower heatsink (2K-FA only)
Temp Combiner	3	Combiner temp (2K-FA only)
Warnings	1	Warning code (N = none)
Alarms	1	Alarm code (N = none)

Warning Codes:

M=Alarm, **A**=No antenna, **S**=SWR, **B**=No band, **P**=Power limit, **O**=Overheat, **Y**=ATU N/A, **W**=Tune no power, **K**=ATU bypass, **R**=Remote hold, **T**=Combiner heat, **C**=Combiner fault, **N**=None

Alarm Codes:

S=SWR limit, **A**=Amp protection, **D**=Overdrive, **H**=Excess heat, **C**=Combiner fault, **N**=None

10. Running as a System Service

To start the server automatically on boot, create a systemd service file:

Step 1: Create the Service File

```
sudo nano /etc/systemd/system/spe-remote.service
```

Paste the following content:

```
[Unit]
Description=SPE Amplifier Remote Control
After=network.target

[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/spe-remote
ExecStart=/home/pi/spe-remote/venv/bin/python server.py
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Adjust User and WorkingDirectory to match your actual username and install path.

Step 2: Enable and Start

```
sudo systemctl daemon-reload
sudo systemctl enable spe-remote
sudo systemctl start spe-remote
```

Step 3: Check Status

```
sudo systemctl status spe-remote
journalctl -u spe-remote -f
```

11. WebSocket API

The server exposes a WebSocket endpoint for custom clients and integrations.

Endpoint:

```
ws://<host>:8888/ws
```

Received JSON (amplifier state):

```
{
  "op_status": "Oper",
  "tx_status": "TX",
  "input": "1",
  "band": "80m",
  "tx_antenna": "1",
  "p_level": "10",
  "p_out": "1353",
  "swr": "1.54",
  "aswr": "1.12",
  "voltage": "54.6",
  "drain": "27.3",
  "pa_temp": "26",
  "warnings": "",
  "error": ""
}
```

Received JSON (power action result):

```
{
  "power_result": "power_on",
  "status": "ok"
}
```

Send Commands (text messages):

Command	Action
power_on	Power ON via DTR toggle
power_off	Power OFF via serial cmd 0x0A
oper	Toggle Operate/Standby
antenna	Cycle TX antenna
input	Toggle input port
tune	Start ATU tuning
power_level	Toggle power level (L/M/H)
band_up	Band up
band_dn	Band down
display	Toggle display
backlight_on	Backlight on
backlight_off	Backlight off

12. Troubleshooting

Problem: Serial error: No such file or directory

Solution: The serial port path in config.yaml does not exist. Check the cable is plugged in and verify the port with: `ls /dev/serial/by-id/`

Problem: Permission denied on serial port

Solution: Add your user to the dialout group: `sudo usermod -aG dialout $USER`, then log out and back in.

Problem: Web page not loading

Solution: Check that the server is running and the port is not blocked by a firewall. Try: `sudo ufw allow 8888/tcp`

Problem: Gauges not updating

Solution: Open browser developer console (F12) and check for WebSocket connection errors. Verify the server IP and port are correct.

Problem: Multiple /dev/ttyUSB devices

Solution: Use the `/dev/serial/by-id/` path which is unique to each USB-serial adapter. This prevents confusion when multiple USB devices are connected.

Problem: Power ON not working

Solution: Verify your FTDI USB-serial adapter supports DTR line control. Check with `dmesg` that the adapter is recognized. The DTR toggle sequence requires hardware DTR support.

Problem: 'POWER SWITCH HELD BY REMOTE' warning

Solution: This is normal behavior when DTR is held high after a remote power on. The amplifier's front-panel power switch is overridden while the remote has control.

Problem: Server crashes on startup

Solution: Check the log output for errors. Common causes: wrong Python version (need 3.9+), missing dependencies (re-run `./setup.sh`), or serial port already in use by another program.

Problem: High CPU usage

Solution: Increase `polling.idle_interval` in config.yaml to reduce serial polling frequency when idle. The default 1.0 second should be fine for most setups.

For additional help, open an issue on the GitHub repository:
<https://github.com/vu2cpl/spe-remote>